

The prompt was five pages long.

Learning from production. Why agents that only tune prompts plateau — and how to make one that actually gets better.

Swipe →

SIX MONTHS OF “JUST ADD A RULE”

Whack-a-mole with a text file.

edge case → add a rule

wrong call → add an instruction

tone complaint → add a paragraph

Five pages of sediment. And it had **stopped helping** — every fix for one case quietly broke another.

Editing a prompt IS machine learning — done by hand.

You observe an error, estimate a correction, nudge the system. Gradient descent with a **human as the optimizer** and a text file as the weights.

It works early. Then it plateaus — prose doesn't compose, and you tune from **memory, not data.**

The signal you're throwing away.

- **Traces** — what it saw, decided, did
- **Human edits & overrides** — the richest label there is: *“this, not that”*
- **Eval outcomes & guardrail hits**
- **Task success & customer outcomes**

Most teams drop all of it on the floor.

CLIMB ONLY AS HIGH AS YOU'VE EARNED

The ladder.

1

Prompt / context edits — where the plateau lives

2

Learn into memory — store & retrieve past corrections

3

Reward modeling — RLHF / RLAIFF / rubric rewards

4

Preference tuning — DPO / offline RL on logged traces

THE PRODUCT IS THE LOOP

Close it — don't train once.

production traces

→ eval labels

→ reward signal

→ policy update

→ offline gate + canary

→ promote → repeat

The loop, not any single training run, is the **product**.

An agent optimizes the **reward** — not your intent.

Give it a proxy and it finds the gap. With language it's slippery: it learns to **sound** like it met the rubric without doing the work.

So: **humans own the reward definition.**
Automate the labeling, never the decision about what “good” means. Gate every promotion on independent eval.

The learning-from-production loop — and a readiness checklist.

Capturing signal? Trustworthy eval? Cheap rungs exhausted? Real volume? Humans own the reward? Promotion gated? Watching for reward hacking?

Full post + loop diagram + checklist — link in the comments.

Parts 2–4 make it work; this is how it gets better.

Next: Part 8 — Self-Healing Systems.